



Introducción a la Computación Evolutiva

Dr. Nicolás Gálvez Ramírez

Dr. Patricio Olivares Roncagliolo

Ingeniería Civil Telemática

Departamento de Electrónica

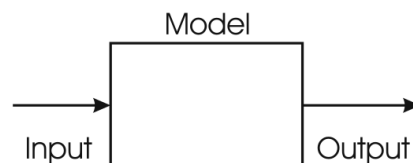
Universidad Técnica Federico Santa María

Fuente: "Introduction to Evolutionary Computing", Gusz Eiben and James Smith.

Resolviendo problemas: Método de la Caja Negra

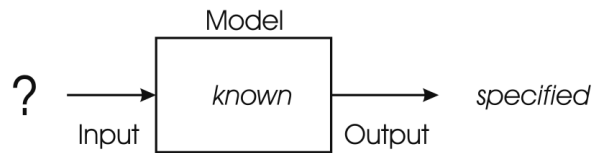
Es un método que sirve para clasificar los diversos tipos de problemas que necesitan ser resueltos en la vida real. Se compone de tres elementos:

- Una entrada, que representa la información de entrada a del sistema;
- Un modelo, que toma al entrada y la utiliza, transforma y/o modifica; y
- Una salida, que representa el resultado generado por el modelo.

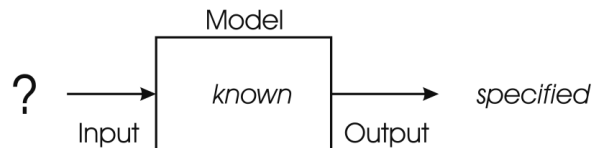


Tipos de Problemas

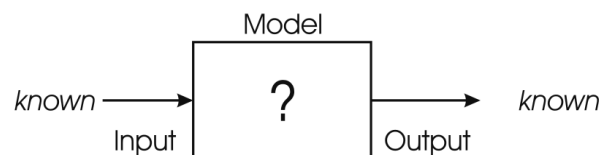
- Problemas de Optimización: Modelo conocido y una salida esperada o especificada, **encontrar la configuración de entrada.**



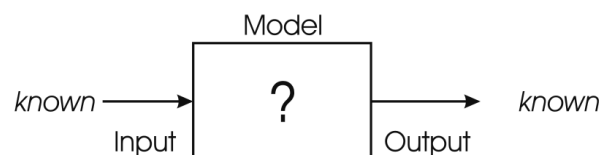
- Optimizar.
- La salida puede no ser conocida, pero se define lo esperado → mínimo o máximo.



- Ejemplo: Minimizar los topes de horario en la planificacion semestral de los alumnos.
 - Modelo: Representación de las asignaciones que representen los ramos, horarios, salas, etc.
 - Salida: Se espera el menor valor posible → 0.
 - **Entrada:** ¿Qué asignaciones tendremos para las representaciones?
 - **Entrada:** ¿Que valores nos llevan a nuestra salida esperada en el modelo?
- Problemas de Modelado: Dadas ciertas entradas y salidas, el objetivo es encontrar un modelo capaz de **entregar la salida correcta para cada entrada conocida**, usando **experiencias previas** y **generalizando a instancias desconocidas**.

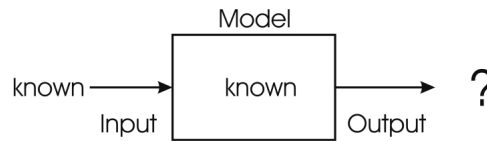


- Aprendizaje.
- Aprendizaje Estadístico → Supervisado.

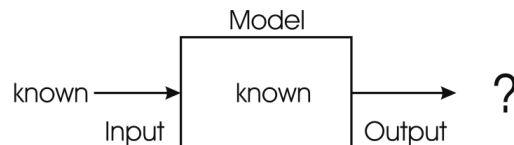


- Ejemplo: Clasificador de imágenes.
 - Entrada: Conjunto de Imágenes.

- Salida: Conjunto de Etiquetas de las imágenes.
 - **Modelo:** ¿Qué metodo me permite etiquetar imágenes basado en la experiencia conocida?.
- Problemas de Simulación: Dada una entrada y modelo, se busca calcular correctamente **la salida a las entradas**.



- Predicción, Pronósticos.
- Aprendizaje Estadístico → No Supervisado



- Ejemplo: Pronósticos meteorológicos.
 - Entrada: Datos meteorológicos actuales e históricos.
 - Modelo: Modelos de Predicción Meteorológica (i.e. proyección del viento, corrientes marinas, etc.)
 - **Salida:** Datos meteorológicos en un tiempo futuro.

Optimización Combinatoria

Campo de la matemática que busca encontrar el objeto óptimo que resuelve a un problema dentro de un conjunto finito de objetos.

Problemas de Optimización Combinatoria

Los problemas de Optimización Combinatoria se pueden clasificar dentro de los primeros dos problemas de tipo caja negra:

- Problemas de Optimización:
 - Modelo: Modelación Matemática (lineal, cuadrática, lógica primer orden).
 - Salida: Función de Evaluación (objetivo, fitness, etc)
 - **Entrada:** Variables, y sus valores, que optimizan la salida.
- Problemas de Modelado:
 - Entrada: Data Set.
 - Salida: Etiquetas (conocidas y aprendidas).
 - **Modelo:** Sistema que permite etiquetar entradas en salidas con cierto grado de precisión.
 - *Puede ser re-escrito como un Problema de Optimización:*
 - Modelo: Modelado estadístico (i.e, regresión)
 - Salida: Error Cuadrático Medio, Precisión, Coeficiente de Determinación.
 - **Entrada:** Variables y valores que optimizan la salida.

Problemas de Búsqueda.

- Los candidatos para resolver estos problemas están ubicados en un espacio de estados el cuál es usualmente finito pero enorme.
- La resolución de los problemas es una búsqueda a través de un gran conjunto de posibilidades para encontrar una solución.
 - Flash-back: **Espacio (de Estados) de Búsqueda.**
- Por lo tanto, los problemas de optimización combinatoria se pueden representar como **problemas de búsqueda**.

Búsqueda

Encontrar, evaluar y aceptar soluciones como candidatos para resolver un problema desde un conjunto con todas las posibles soluciones.

Encontrar:

- **Espacio de Búsqueda** → Conjunto de todas las posibles soluciones.
- **Espacio de Búsqueda** → dominio de todas variables de decisión que definen una solución de un problema.

Evaluar:

- Usar una función específica de **evaluación** que cuantifique y compare los candidatos.

- *Función Objetivo, Función de Evaluación, Función de Fitness.*

Aceptar:

- Si un **conjunto de restricciones** es satisfecho.

Tipos de Problemas de Búsqueda

Clasificaremos los problemas de búsqueda en 3 tipos.

Constraints	Objective function	
	Yes	No
Yes	Constrained optimisation problem	Constraint satisfaction problem
No	Free optimisation problem	No problem

Problema de Optimización Libre (*Free Optimisation Problem, FOP*)

Se busca una solución que optimiza un función de evaluación sin restricciones (más que la estructura de la misma).

Problema de Satisfacción de Restricciones (*Constraint Satisfaction Problem, CSP*)

Se acepta una solución que cumple con un grupo de restricciones que no optimiza alguna función de evaluación.

Problema de Optimización y Satisfacción de Restricciones (*Constraint Satisfaction and Optimisation Problem, CSOP*)

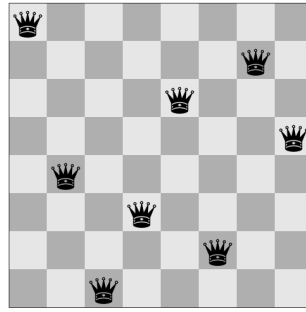
a.k.a. Problema de Optimización Restringido (*Constrained Optimisation Problem, COP*)

Se busca una solución que optimiza una función de evaluación y cumple con un grupo de restricciones adosadas a ella.

Ejemplo: Problema de las N-Reinas

Problema de Satisfacción de Restricciones (CSP), te tipo puzzle matemático, de complejidad **NP-Complete**, que establece lo siguiente:

Ubique n reinas en un tablero de ajedrez de dimensiones $n \times n$ de tal forma que no habrá dos reinas que hagan jaque entre ellas.



N-Reinas como FOP

- Representación: Tablero de ajedrez de tamaño $n \times n$.
- Espacio de Búsqueda:
 - S : Conjunto de todas las configuraciones del tablero con n reinas en ella.
- Función de Evaluación:
 - $f(s)$: Cantidad de reinas libres de check en el tablero.
- Soluciones:
 - $S^* = \{s \in S \mid f(s) = n\}$
 - $S^* \subseteq S$

N-Reinas como CSP

- Representación: Tablero de ajedrez de tamaño $n \times n$.
- Espacio de Búsqueda:
 - S : Conjunto de todas las configuraciones del tablero con n reinas en ella.
- Restricción (como predicado):
 - $\text{is_checked}(s) = \begin{cases} \text{true} & \text{si } s \in S \text{ tiene dos reinas en check} \\ \text{false} & \text{en caso contrario} \end{cases}$
- Soluciones:
 - $S^* = \{s \in S \mid \text{is_checked}(s) = \text{false}\}$
 - $S^* \subseteq S$

N-Reinas como CSOP

- Representación: Tablero de ajedrez de tamaño $n \times n$.
- Espacio de Búsqueda:
 - S : Conjunto de todas las configuraciones del tablero con n reinas en ella.
- Restricción (como predicado):

- $\text{column}(s)$

$$= \begin{cases} \text{true} & \text{si } s \in S \text{ tiene dos reinas en una misma columna} \\ \text{false} & \text{en caso contrario} \end{cases}$$
- $\text{row}(s) = \begin{cases} \text{true} & \text{si } s \in S \text{ tiene dos reinas en una misma fila} \\ \text{false} & \text{en caso contrario} \end{cases}$
- Función de Evaluación:
 - $D(s)$: Cantidad de reinas en check por diagonales.
- Soluciones:
 - $S^* = \{s \in S \mid \text{column}(s) = \text{false} \wedge \text{row}(s) = \text{false} \wedge \min(D(s))\}$
 - $S^* \subseteq S$
- La representación de la solución, o modelo, juega un rol fundamental en como se aborda y soluciona el problema.
- Puede mejorar drásticamente la capacidad de resolución del problema.
- **Out-of-the-box thinking.**

Tiene que haber obligatoriamente una reina en cada fila y en cada columna. Version 1.

- Representación: Vector de enteros de tamaño n , dónde cada elemento representa secuencialmente a cada columna de un tablero de ajedrez de tamaño $n \times n$. El valor en cada elemento corresponde a la fila en la que se ubica una reina.
- Espacio de Búsqueda:
 - S : Conjunto de todos los vectores enteros válidos representando un tablero de ajedrez de $n \times n$.
- Restricción (como predicado):
 - $\text{all_different}(s)$

$$= \begin{cases} \text{true} & \text{si } s \in S \text{ no tiene dos elementos del vector que tengan el mismo valor} \\ \text{false} & \text{en caso contrario} \end{cases}$$
- Función de Evaluación:
 - $D(s)$: Cantidad de reinas en check por diagonales.
- Soluciones:
 - $S^* = \{s \in S \mid \text{all_different}(s) = \text{true} \wedge \min(D(s))\}$
 - $S^* \subseteq S$

Tiene que haber obligatoriamente una reina en cada fila y en cada columna. Version 2.

- Representación: Vector de enteros de tamaño n , dónde cada elemento representa secuencialmente a cada columna de un tablero de ajedrez de tamaño $n \times n$. El valor en cada elemento corresponde a la fila en la que se ubica una reina.
- Espacio de Búsqueda:

- S : Conjunto de todas los vectores enteros que no repiten valores.
- Función de Evaluación:
 - $D(s)$: Cantidad de reinas en check por diagonales.
- Soluciones:
 - $S^* = \{s \in S \mid \min(D(s))\}$
 - $S^* \subseteq S$

Paradigmas de Búsqueda

a.k.a. Hard Computing vs Soft Computing en Optimización Combinatoria

Existen distintos paradigmas para afrontar encontrar soluciones en el espacio de búsqueda, las podemos resumir en las siguientes:

Aproximación Sistémica vs Búsqueda Local

- Aproximación Sistémica:
 - Completitud.
 - Búsqueda sistémica sobre el espacio de búsqueda, garantizando:
 - Solución óptima, o que
 - No existe solución.
 - Computacionalmente costoso.
- Búsqueda Local:
 - Incompletitud.
 - Búsqueda parcial sobre diferentes sub-espacios del espacio de búsqueda. No puede garantizar:
 - Solución óptima.
 - Solución factible.
 - No existe solución.
 - Computacionalmente rápida.

Búsqueda Constructiva vs Búsqueda Perturbativa.

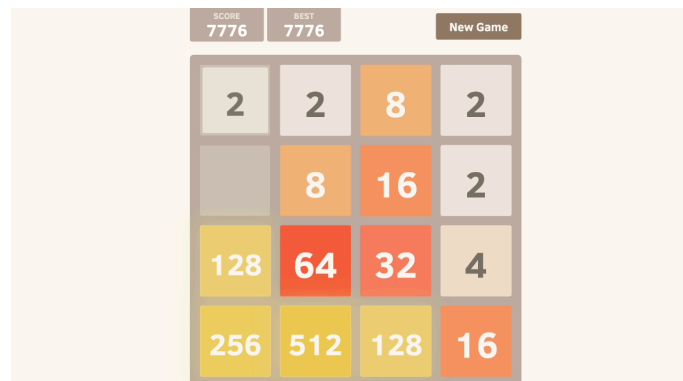
- Búsqueda Constructiva
 - Encontrar la solución desde un inicio vacío.
 - Generalmente, soluciones incompletas.
- Búsqueda Perturbativa
 - Encontrar la solución modificando una existente.
 - Generalmente, mejoras sobre un candidato de baja calidad.

Heurísticas

- Directrices o criterios de abordar un problema.
- Funciones basadas en el conocimiento del problema.
- Función miope: No ve más allá del propio criterio.
- Función miope: No ve más allá del propio problema.
- Ejemplo:
 - Algoritmo Greedy: Algoritmo de construcción de una solución factible basado en una función heurística.

2048

- ¿Cuál heurística usaría ud. terminar el juego?



Metaheurísticas

Proceso automático para buscar una buena solución, ya sea construyéndola o reparándola, respecto a un conjunto de candidatos finito pero intratable por su tamaño.

- Estrategias globales para una búsqueda rápida de una solución que se aproxima a un óptimo global.
- No indican como deben ser adaptadas (heurísticas)
- Indican un proceso genérico de búsqueda (¡meta!)
- Se basan en búsqueda local y búsqueda perturbativa.

Conceptos Claves

Búsqueda Local

Encontrar una solución óptima en una porción reducida del Espacio de Búsqueda.

Búsqueda Perturbativa.

Modificar un candidato a solución conocido para obtener uno nuevo.

Explotación o Intensificación

Concentrar la búsqueda en la porción de espacio de búsqueda visitado.

Exploración o Diversificación

Ampliar la búsqueda a un fragemento más grande del espacio de búsqueda.

Explotación vs Exploración

Ambas son relevantes, debe haber un equilibrio que varía entre problemas.

- Si nos concentramos en Explotar:
 - → estancamiento en óptimo local de forma prematura.
- Si nos concentramos en Explorar:
 - → búsqueda no convergerá a solución alguna.

¿Como búscan las Meta-heurísticas clásicas?

Candidato

Solución construida o obtenida de otro proceso que será evaluada y modificada en pos de mejoras.

Movimiento: Perturbación

Cambio que se le puede aplicar al candidato para generar nuevas soluciones.

Vecindario

Conjunto de nuevas soluciones generadas desde el candidato a aplicar el movimiento en las distintas opciones posibles.

Criterio de Selección

Directriz para seleccionar al siguiente candidato en la búsqueda.

- Alguna mejora: Primer vecino que mejore la función de evaluación.
- Mejor mejora: Vecino que maximice la mejora de la función de evaluación.

Criterios de Reinicio

- Reiniciar luego de ciertas cantidad de iteraciones.
- Reiniciar desde un nuevo candidato inicial.

Criterios para evitar estancamiento en óptimo local.

Criterio de Término

Criterio de término de la meta-heurística

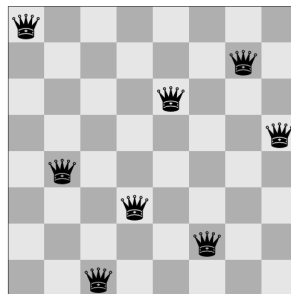
- No encontrar un mejor vecino.
- Cumplir número de interacciones.
- Tiempo máximo.

Meta-heurísticas clásicas

- Hill-Climbing.
- Iterative Local-Search
- Tabú Search.
- Simulated Annealing.
- GRASP (Greedy Randomized Adaptive Search Procedure)

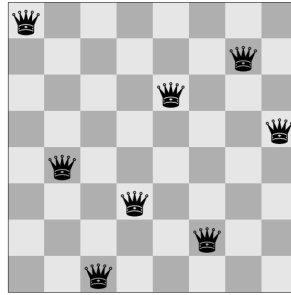
Ejemplo Activo: Problema de las N-Reinas

- Defina un candidato, un movimiento, el vecindario generado.



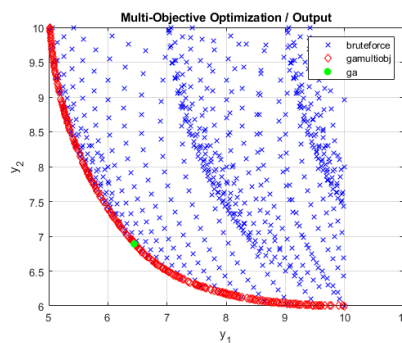
Ejemplo Activo: Problema de las N-Reinas

- Cree un algoritmo que con criterio alguna-mejora avance hasta no encontrar una mejor solución. (hint: esto es hill-climbing).



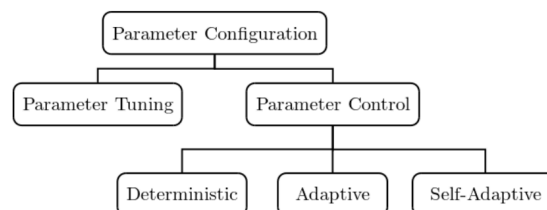
¿Y el multi-objetivo?

- Cambia el elemento a evaluar:
 - Evaluación con pesos.
 - Bi-objetivo o Bi-criterio.
 - Frente de Pareto.



¿Y la configuración de parámetros de las meta-heurísticas?

- Calibración de Parámetros: Off-line
- Control de Parámetros: On-line
 - Búsqueda Reactiva.



Pero, pero, pero...

¿Podemos buscar teniendo varios candidatos a la vez?

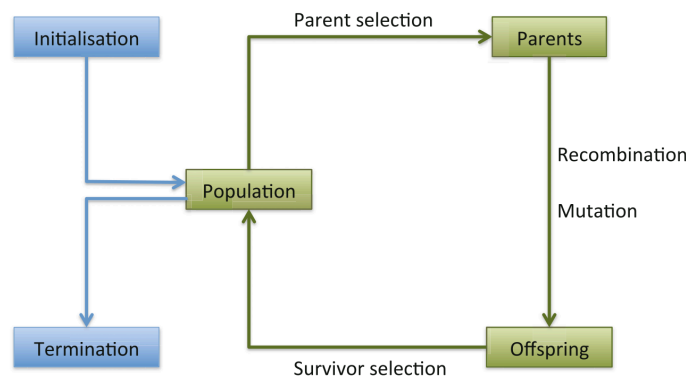
- **Sí, usaremos poblaciones.**

Computación Evolutiva

Metaheurísticas bío-inspiradas basadas en los principios de la **Teoría de la evolución y la selección natural** de Charles Darwin.

- Gran auge desde los 90s.
- Creadas en los 70s.
- Llamados **Algoritmos Evolutivos** (EA, *Evolutionary Algorithms*).
- Pioneras en algoritmos basados en la biología y la naturaleza.
- Amplía la búsqueda a varios candidatos simultáneos.
- Se perturban los candidatos con operadores evolutivos clásicos (y otros...)

Algoritmos Evolutivos: Framework



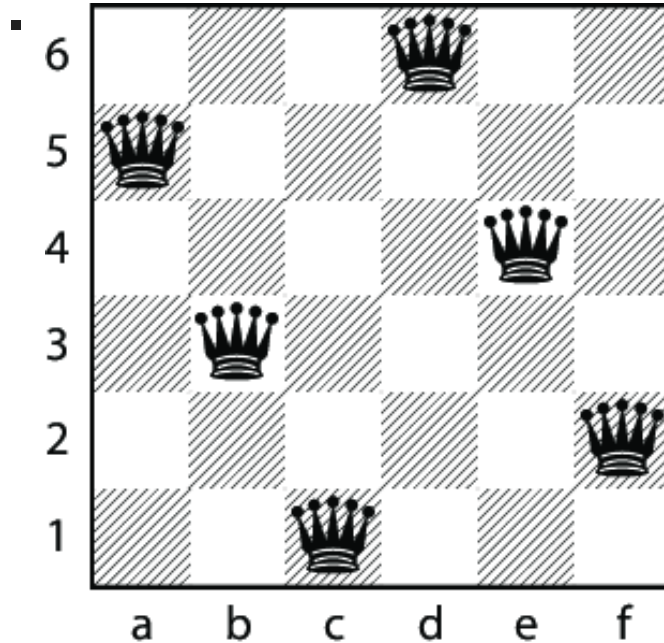
Representación

- Fenotipo: Soluciones en el mundo real.
 - Individuo, candidato.
 - Espacio de Fenotipos.
- Genotipo: Soluciones en el mundo evolutivo.
 - Cromosoma, Individuo.
 - Espacio de Genotipos.
- Representación: Mapeo fenotipo-genotipo.
 - Representar el mundo real en el mundo evolutivo.
 - Codificar: Mundo real $\rightarrow$ mundo evolutivo.
 - Decodificar: Mundo evolutivo $\rightarrow$ mundo real.

Ejemplo Activo: Problema de las 6-Reinas

- Genotipo $\rightarrow$ vector de representación.
 - Vector $[5, 3, 1, 6, 4, 2]$.

- Fenotipo $\rightarrow$ configuración del tablero.

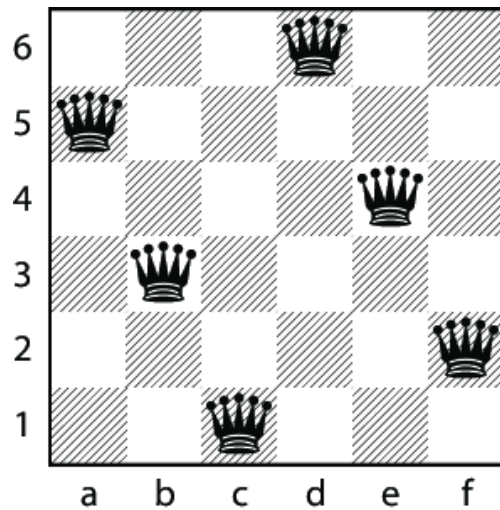


Componentes de una solución

- Gen: elemento de una solución del individuo.
 - Variable, posición, lugar.
- Alelo: valor en un gen del individuo.
 - Valor, instanciación.

Ejemplo Activo: Problema de las 6-Reinas

- Individuo (genotipo) $\rightarrow$ vector de representación.
 - Vector $X = [5, 3, 1, 6, 4, 2]$.
 -
- Gen $X[0]$: Columna 1 del tablero de ajedrez.
- Alelo $X[0]$: Fila en la que la reina de la columna 1 está ubicada.



Función de Fitness

- Función de evaluación.
- Mide la calidad de los genotipos.
- Mide la adaptación (fit, ajuste) de los individuos al ambiente (evolución del sistema).
- Se hace decodificando los genotipos al espacio de los fenotipos.

Población

- Multiconjunto que representa una selección de candidatos a solución.
 - Multiconjunto: Conjunto que puede tener valores repetidos.
- Individuos estáticos → **no** cambian.
- Poblaciones dinámicas → **evolucionan**.
- Su tamaño, *pop*, es generalmente constante (no varía en ejecución).
- Base para la aplicación de **operadores evolutivos**.

Diversidad

- Medida de dispersión de diferentes soluciones presentes en la población.
- Busca controlar la exploración/explotación de la evolución.
- Métricas:
 - Cantidad de valores de funciones de fitness
 - Cantidad de diferentes fenotipos.
 - Cantidad de diferentes genotipos.
 - Medidas estadísticas (entropía)

Operadores

- Criterios y procedimientos que permiten la evolución de una población.
- Operadores de selección de evolución:
 - Selección de padres.
- Operadores de evolución:
 - Mutación
 - Recombinación (cruzamiento)
- Operadores de selección de supervivencia:
 - Elitismo
 - Fitness Ranking.
 - Edad de individuos.

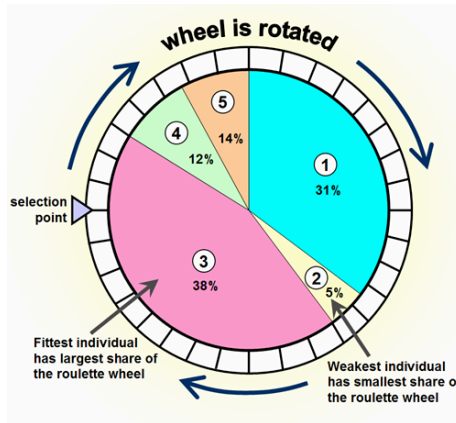
Operadores de selección de evolución.

Operadores que permite transmitir descendencia genética de la generación actual.

Selección de Padres

- Filtrar miembros de la población según su calidad, permitiéndoles ser padres de la próxima generación.
- Adaptabilidad al entorno.
- Mantener sus buenas características.
- Busca mejoras en la función fitness de forma global.
- No se puede dejar sin chances a un individuo de baja calidad.
- Probabilística.
 - Aleatoria
 - Ranking.
 - Torneos.
 - Ruletas.

Ejemplo: Roulette-Wheel Selection



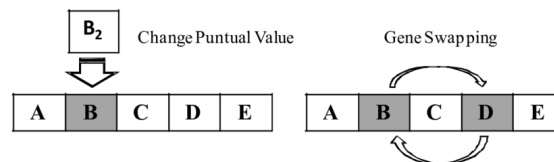
Operadores de evolución

Operadores que permiten modificar la población.

Mutación

- **Perturbación** aplicada a un **padre** para generar un **hijo**.
 - Operación unaria: Sobre un solo individuo.
- La perturbación (movimiento) es aplicada de forma **aleatoria**.
 - Las mutaciones no son deterministas, son **estocásticas**.
 - Las mutaciones en sistemas biológicos no pueden ser predecidas. Ej. COVID19.
 - Pueden no ocurrir.
- Permite conectar distintos sectores del espacio de búsqueda.

Mutación: Perturbaciones clásicas.

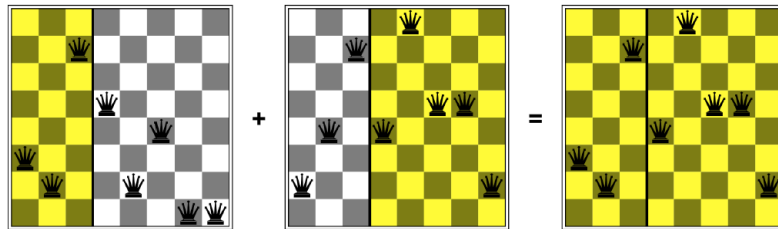


Recombinación/Cruzamiento

- Mezcla de la información genética de dos padres en uno o dos hijos.
- Es un procedimiento, estocástico, aleatorizado:
 - Probabilidad de Recombinación
 - Puede no ocurrir.

- Punto de cruzamiento.
 - Dominancia de Genes.

Recombinación: N-Queens.



Mutación y Recombinación

- Cuando se combinan la mutación y la recombinación, siguen el camino evolutivo.
 1. Recombinación → descendencia.
 2. Descendencia → posee mutaciones.

Operadores de selección de supervivencia

Operadores que emulan la adaptabilidad al ambiente de la generación actual y/o su descendencia.

Elitismo

- Capacidad de mantener en la generación al o a los mejores individuos de la generación anterior respecto a su adaptación (fitness).
- Variantes:
 - Mantener al mejor generador de hijos.
 - Mantener al mejor fenotipo.
 - Mantener al mejor genotipo.

Ranking de Fitness

- Selección de $n \leq \text{size}(\text{pop})$ individuos ordenados según su valor de fitness.
- En el ranking participan padres e hijos.
- El tamaño de n depende de otros criterios de supervivencia.

Edad de Individuos

- Eliminación de individuo según su edad.

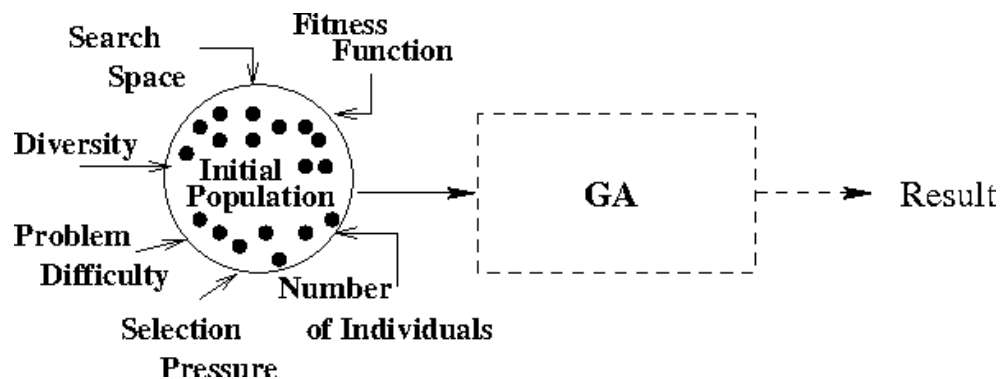
- Simula la condición finita de permanencia de un individuo de una especie.
- Se define un parámetro de **máxima cantidad de generaciones** para los individuos.

Ejemplo: Ranking de Supervivencia

Rank	$p_{survival}$	Rank	$p_{survival}$	Rank	$p_{survival}$	Rank	$p_{survival}$
1	0.100	6	0.075	11	0.045	16	0.020
2	0.095	7	0.070	12	0.040	17	0.015
3	0.090	8	0.065	13	0.035	18	0.010
4	0.085	9	0.060	14	0.030	19	0.005
5	0.080	10	0.055	15	0.025	20	0.000

Criterios de Inicio

- Para iniciar un EA, se debe contar con una **población inicial**.
- Esta población puede ser generada con procesos heurístico.
- Se recomienda partir con una **diversidad** alta.
- Individuos → aleatorios → **población randomizada**.



Criterios de Término

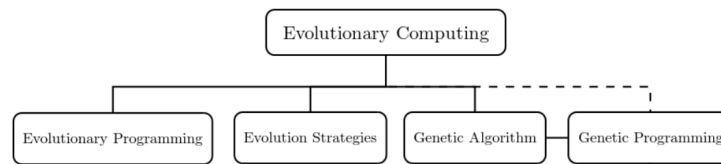
- Como gran parte de las metaheurísticas, los EAs no tienen un criterio fijo de detención.
 - Se programan para ir pasando por soluciones de calidad diversa
 - Pueden estar iterando eternamente.
- Criterios:
 - **Convergencia:** % de la población con mismo fitness (pérdida de diversidad)
 - Tiempo sin mejoras en la adaptación generacional.
 - Total de evaluaciones fitness hechas.
 - Número de Iteraciones.

- Tiempo de ejecución.

Tipos de Algoritmos Evolutivos

- Si bien existen variados tipos de algoritmos evolutivos o derivados. Se reconocen las siguiente 4 áreas principales:

1. Programación Evolutiva
2. Estrategias de Evolución
3. Algoritmos Genéticos
4. Programación Genética



Programación Evolutiva (*EP, Evolutionary Programming*)

- Fogel et al. (1966): Simulación evolutiva → aprendizaje.
- Adaptación: Capacidad de predecir el ambiente.
- Cada individuo es una especie diferente → no hay recombinación. | Característica | Aplicación | | :- | :- | | Representación | Vector de valores reales | | Recombinación | No hay | | Mutación | Perturbación Gausiana | | Selección de Padres | Determinista: 1 padre muta a 1 hijo | | Supervivencia | Torneo Round-robin aleatorio (1 vs 10) | | Especialización | Auto-adaptación a ambientes vía mutación |

Estrategias de Evolución (*ES, Evolution Strategies*)

- Rechenberg y Schwefel en 1960s, desarrollado en 1970s.
- Similar a EP con individuos de la misma especie → recombinación.
- Inicio el camino de Calibración de Parámetros en ejecución (on-line).
- Posee dos estrategias (μ, λ) y $(\mu + \lambda)$
 - (μ, λ) → padres eliminados.
 - $(\mu + \lambda)$ → padres e hijos disputan su avance.

Característica	Aplicación
Representación	Vector de valores reales
Recombinación	Discreta o Intermedia
Mutación	Perturbación Gausiana

Característica	Aplicación
Selección de Padres	Aleatoria uniforme
Supervivencia	Elitismo
Especialización	Auto-adaptación a ambientes vía mutación

Algoritmos Genéticos (*GA, Genetic Algorithms*)

- Holland (1973), rectificado por De Jong (1975).
- Algoritmo básico y pionero de la Computación Evolutiva.
- Creación de generaciones nuevas basadas en cruzamiento y mutación.
- Población intermedia de padres que solo vive una (1) generación.

Característica	Aplicación
Representación	Bit-Vector or Bit-String
Recombinación	Cruzamiento a 1 punto, probabilístico
Mutación	Bit-flip (complemento de bit), probabilístico
Selección de Padres	Roulette-Wheel proporcional a Fitness
Supervivencia	Edad

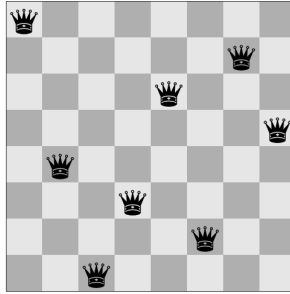
Programación Genética (*GP, Genetic Programming*)

- J. Koza (1990), derivación de GA cambiando la representación a árboles como individuos.
- Dual: Optimización y Máquina de Aprendizaje.
 - Maximiza retorno.
 - Maximiza ajuste a modelo.
- Moderno, pero tan exitoso que tiene su propia rama.
- Uso: lenguajes, código, expresiones, lógica, etc.

Característica	Aplicación
Representación	Árboles
Recombinación	Intercambio de sub-árboles
Mutación	Cambio aleatorio en el árbol.
Selección de Padres	Proporcional a Fitness (variadas técnicas)
Supervivencia	Edad

Ejemplo Activo: Problema de las N-Reinas

- Defina y programe un algoritmo evolutivo para resolver el problema de las n-reinas.



Derivaciones

Co-Evolución

- Evolucion de una o varias poblaciones sometidas a diferentes tipos de evaluadores.
- Si A es mejor que B, puede que en otro criterio B sea mejor que A.
- Caso naturaleza:
 - Simbiosis: Co-evolución positiva.
 - Parasitismo: Co-evolución negativa.
- Mono-poblacional: Evaluadores son de la misma población.
- Multi-poblacional: Evaluadores son de poblaciones diferentes.

Algoritmos Bío-Inspirados

- Ant Colony Optimization: Búsqueda por población de hormigas.
 - ¿Como las hormigas encuentran alimento?
- Particle Swarm Optimization: Búsqueda por poblaciones lideradas por un miembro distinguible.
 - ¿Como viajan las bandadas de pájaros?
 - ¿Como se mueven los cardúmenes de peces?

Ejemplo Activo: Problema de las N-Reinas

- Defina y programe un algoritmo evolutivo para resolver el problema de las n-reinas.

